The wallet's response is a `Wallet.Response` wrapped in a `RequestWalletFunds` command, which is subsequently processed by the command handler. It is processed by `validateFunds`, which persist the event or fails:

```
object Bet {
  ...

  private def validateFunds(
      state: OpenState,
      command: RequestWalletFunds): Effect[Event, State] = {
    command.response match {
      case Wallet.Accepted =>
        Effect.persist(FundsGranted(state))
      case Wallet.Rejected =>
        Effect.persist(
          Failed(state.status.betId, "funds not available"))
    }
  }
  ...
}
```

Now let's move on to the settlement.

## Handling commands: Settle

When a sporting event is over, the application must settle all bets to determine which bets won. What you see here is from the bet's point of view when it receives the `Settle` with the market information, uses it to check the result of the sporting event, and, if it has won, returns the profit to its wallet.

**NOTE** You could argue that it would be better and more reliable to have a blank `Settle` message that forces the bet to request the result from the market to get the information directly from the source. On the other hand, each bet would have to contact the market, which